今天的内容

- 流水线-继续

今日及接下来几次课程的内容安排

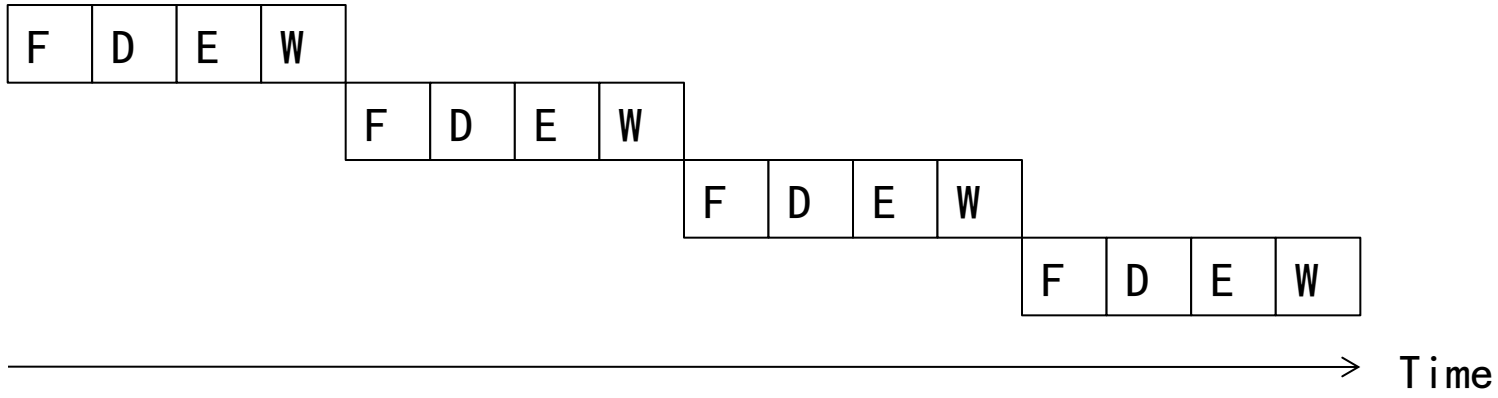
- 单周期微体系结构（Single-cycle Microarchitectures）
- 多周期与微程序化微体系结构（Multi-cycle and Microprogrammed Microarchitectures）
- 流水线技术（Pipelining）
- 流水线中的问题：控制与数据相关性处理、状态维护与恢复等（Issues in Pipelining: Control & Data Dependence Handling, State Maintenance and Recovery, …）

回顾：流水线-基本思想

- **更系统地说（More systematically）：**
  - 将多条指令的执行过程进行**流水化（pipeline）**。
  - 类比：指令的“装配线式处理（Assembly line processing）”。
- **核心思想（Idea）：**
  - **将指令处理周期划分为若干独立的“阶段”（stages）。**
  - 确保每个阶段拥有足够的硬件资源，以便在每个阶段能同时处理一条指令。
  - **在每个阶段处理不同的指令。**
    - 程序顺序上相邻的指令在流水线中被处理于连续的阶段。
- **优点（Benefit）：**
  - 提高指令处理吞吐率（instruction throughput），即增加每周期执行指令数（1/CPI）。
- **缺点（Downside）：**
  - 思考流水线可能带来的问题……

回顾：执行四条相互独立的加法指令

- 多周期（Multi-cycle）：每条指令需要 4 个周期



- 流水线（Pipelined）：4 个周期执行 4 条指令（稳定状态下）

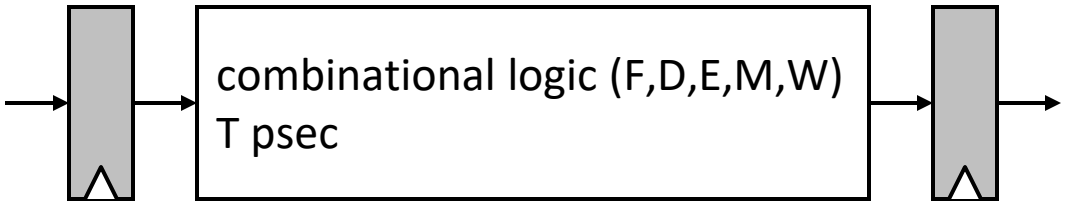


生活总是这么美好吗？

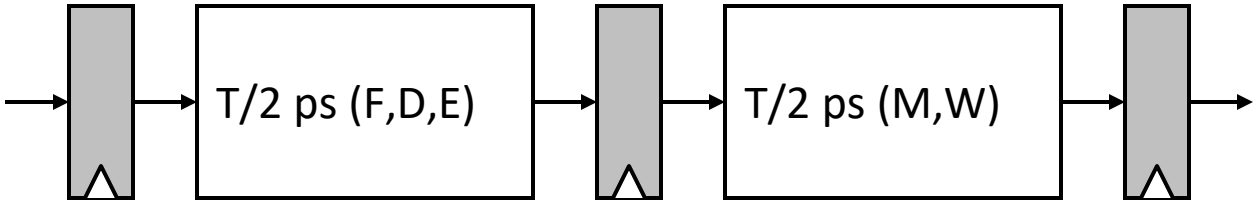
回顾：理想的流水线（An Ideal Pipeline）

- **目标（Goal）：以**极少的成本增加**来提升吞吐率。（在指令处理的情况下，成本指硬件开销。）**
- **重复执行相同操作**
  - 同一个操作在大量不同输入上重复执行。（例如：所有洗衣任务都经历相同的步骤。）
- **重复执行相互独立的操作**
  - 不同操作之间不存在依赖关系。
- **可均匀划分的子操作**
  - 处理过程可以被均匀地划分为多个具有相同延迟的子阶段，且这些阶段**不共享资源**。
- **典型示例：**
  - 汽车装配线、洗衣任务。
  - 那么，对于**指令处理周期（instruction processing “cycle”）**，是否也可以这样实现呢？

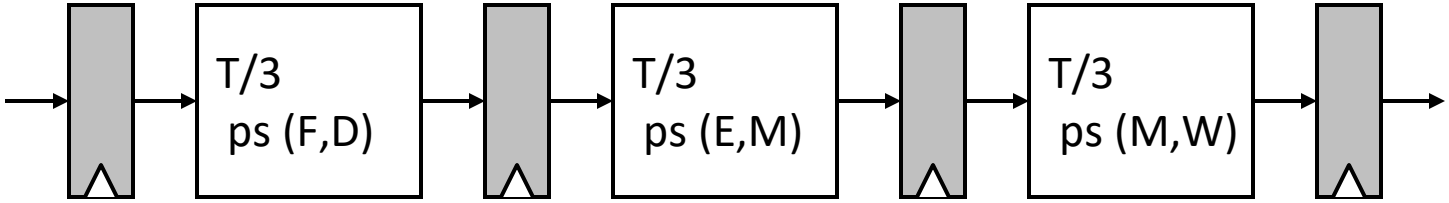
回顾：理想化的流水线



BW=~(1/T)



BW=~(2/T)



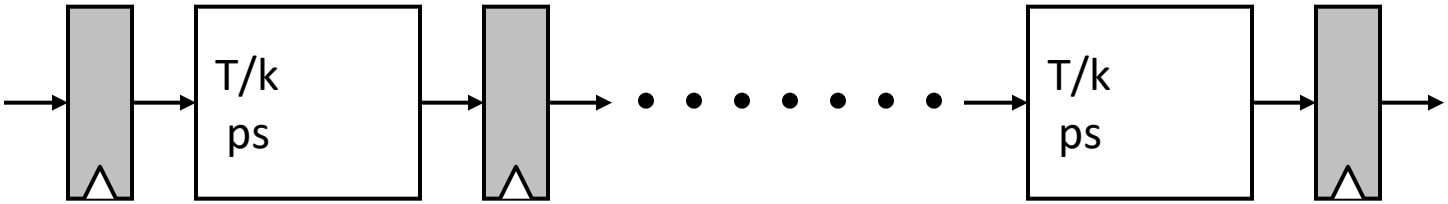
BW=~(3/T)

回顾：更现实的流水线-吞吐率

- 非流水线版本（Nonpipelined version）延迟为 T
  - $BW = 1/(T + S)$
  - 其中 S = 锁存器延迟（latch delay）。



- k 级流水线版本（k-stage pipelined version）
  - $BW_{k-stage} = 1/(T/k + S)$
  - $BW_{max} = 1/(1\ gate\ delay + S)$
  - 锁存器延迟会降低吞吐率
    （阶段之间的切换开销会带来性能损失）

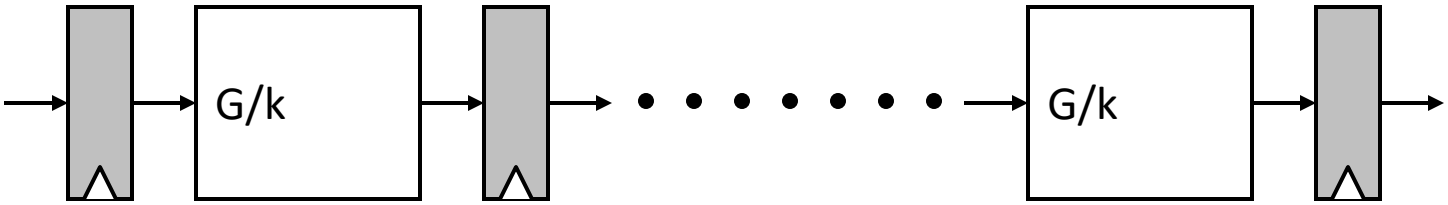


回顾：更现实的流水线-成本

- 非流水线版本（Nonpipelined version）组合逻辑成本为 G
  - Cost = $G + L$ 其中 L = 锁存器成本（latch cost）。

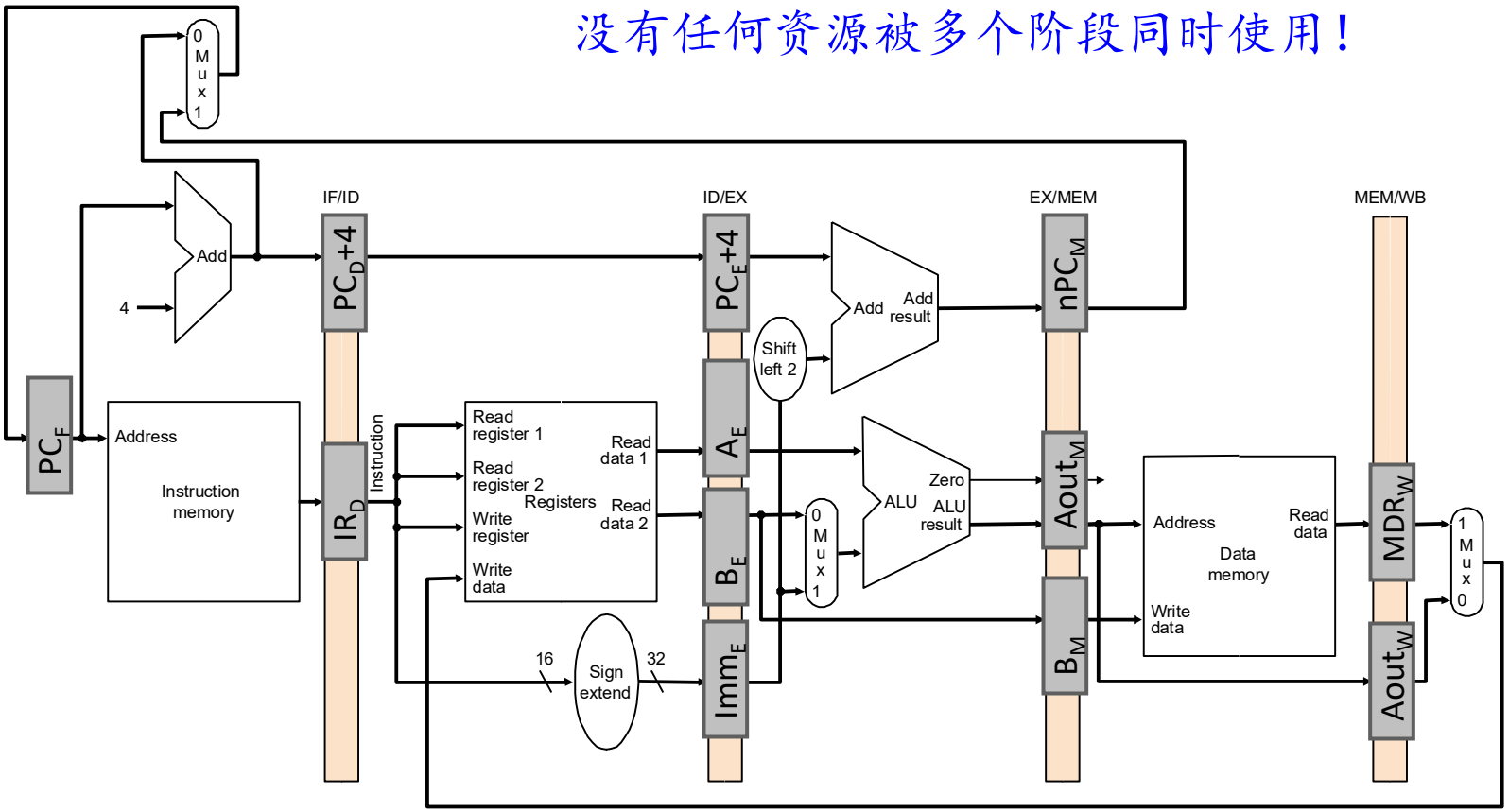


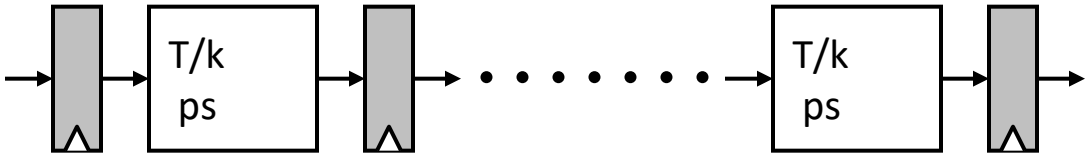
- k 级流水线版本（k-stage pipelined version）
  - $\text{Cost}_{k-stage} = G + Lk$
  - 锁存器的数量随流水线级数增加，从而提高硬件成本。



回顾:实现流水线处理–流水线寄存器

没有任何资源被多个阶段同时使用！



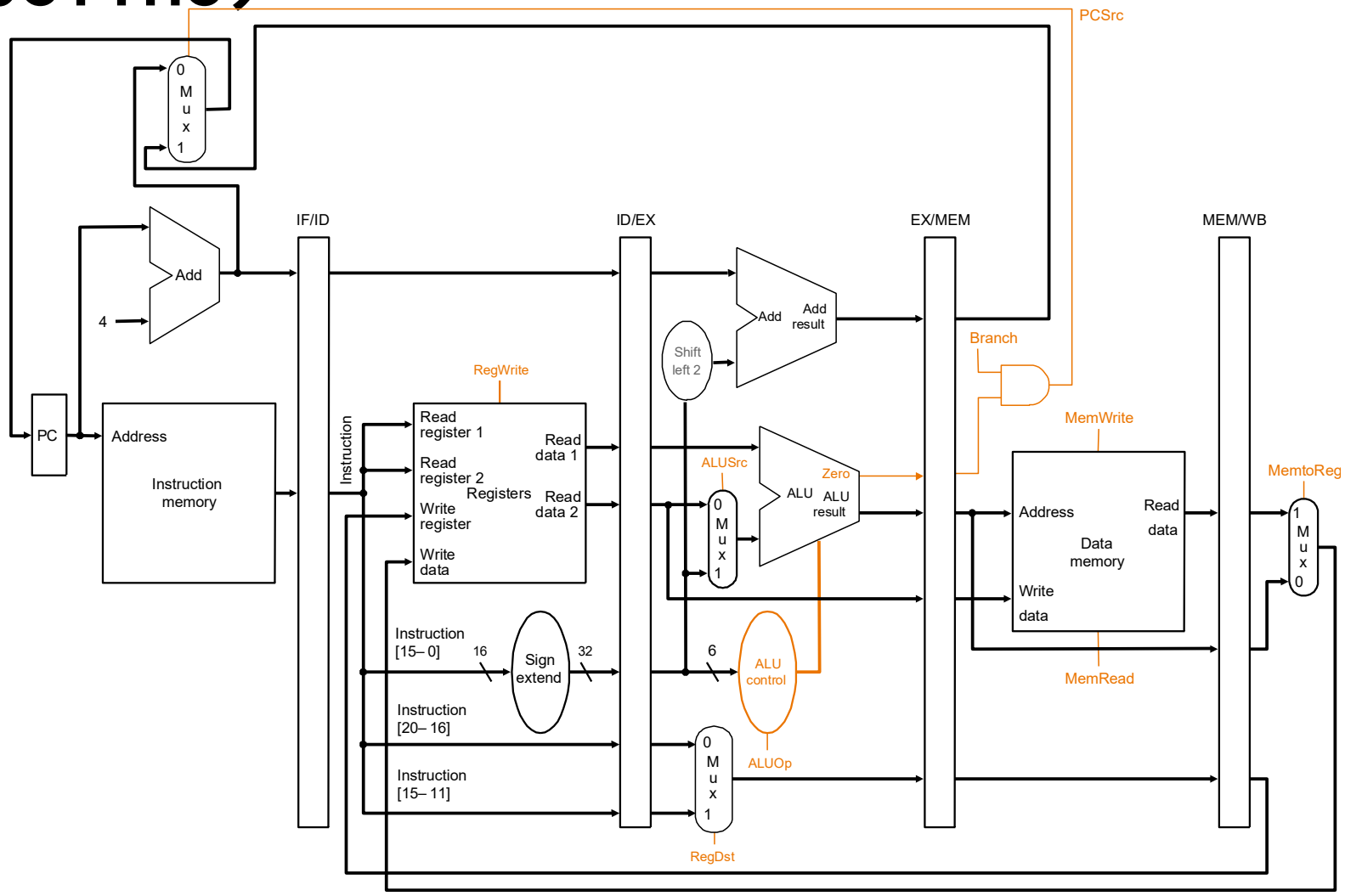


回顾：流水线操作示意-资源视图

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
IF	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}
ID		I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9
EX			I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8
MEM				I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
WB					I_0	I_1	I_2	I_3	I_4	I_5	I_6

在稳态下，处理器的所有硬件单元都被充分利用。

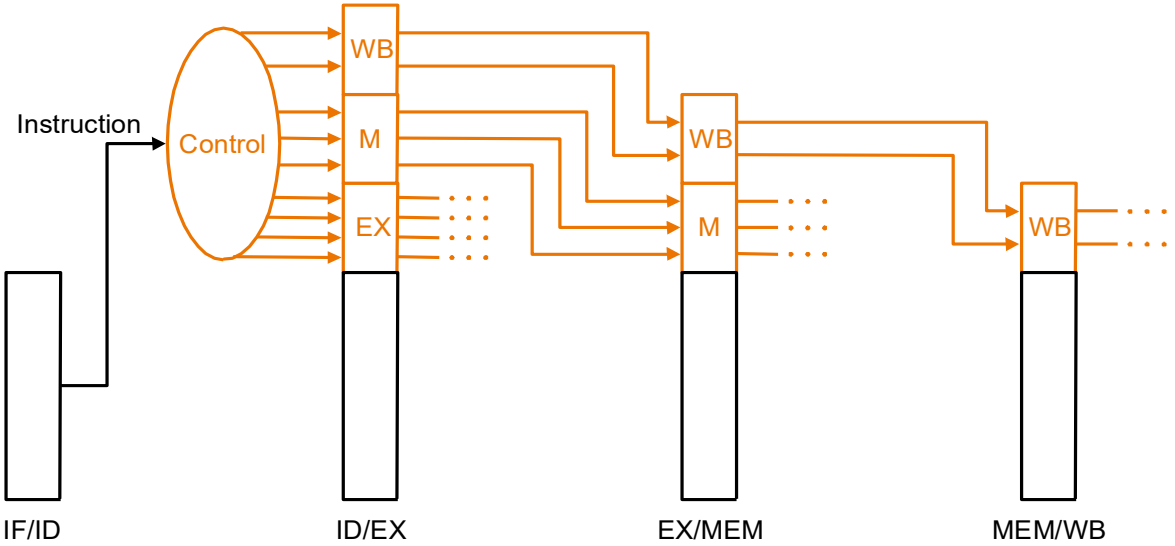
流水线中的控制点（Control Points in a Pipeline）



- 控制点与单周期数据通路完全相同！

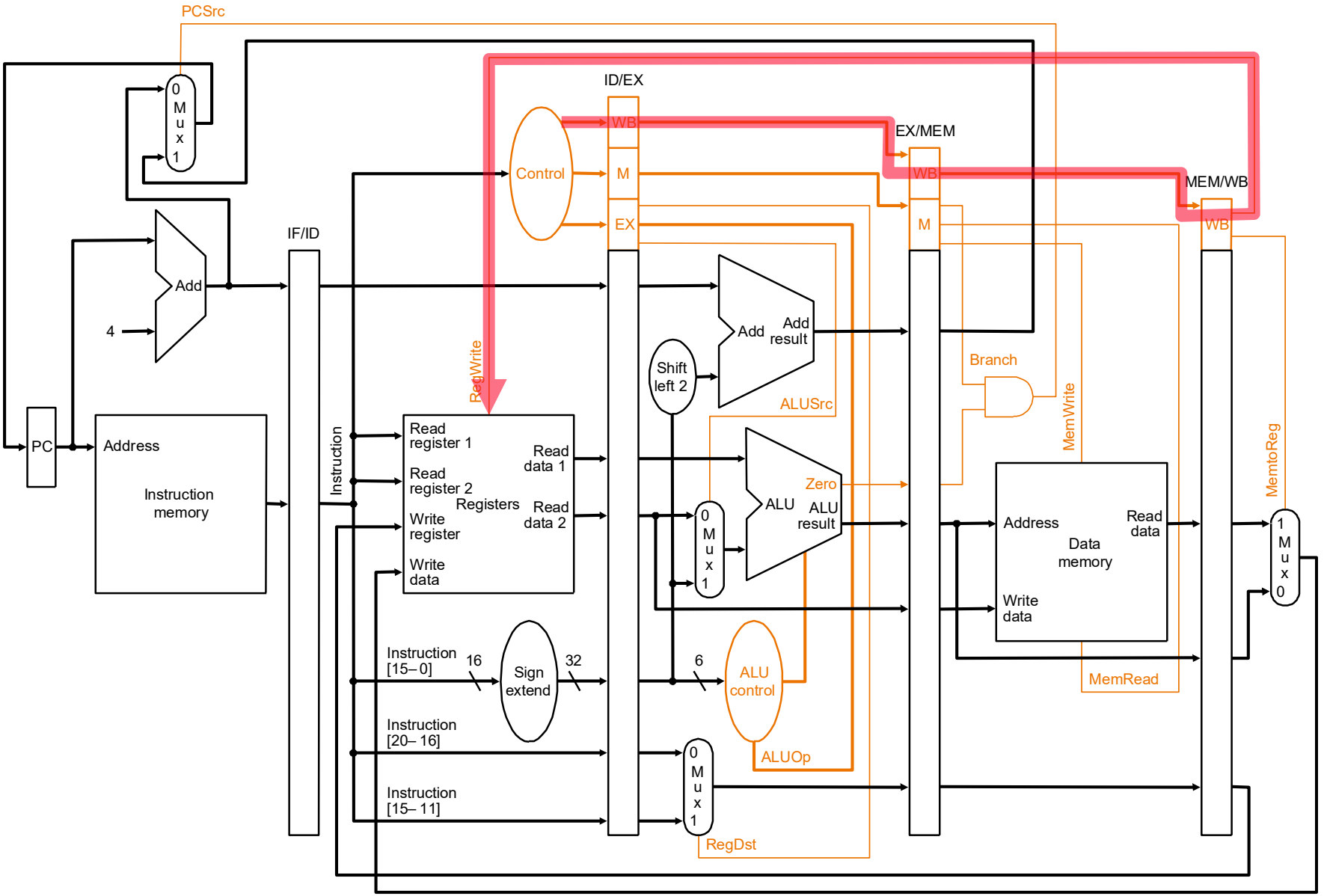
流水线中的控制信号

- **对于给定的指令：**
  - 使用与单周期相同的控制信号，但是
  - 控制信号在不同周期中需要不同的时序，具体取决于阶段。
  - 选项 1：使用与单周期相同的逻辑解码指令，并在信号被消费之前将信号缓冲。这种方法中，控制信号会在多个流水线阶段之间传播，并持续直到该阶段需要它们。



  - 选项 2：将相关的“指令字/字段（instruction word/field）”传递到流水线的每个阶段，并在每个阶段或之前的阶段内解码。这种方法在每个阶段只使用必要的控制信号，避免了冗余。
  - 哪种方式更好？（Which one is better?）

流水线中的控制信号


PCSrc
0
M
u
x
1
ID/EX
EX/MEM
MEM/WB
WB
M
EX
Control
Add
4
IF/ID
PC
Address
Instruction memory
Instruction
RegWrite
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALUSrc
Branch
MemWrite
MemtoReg
Zero
ALU
ALU result
Address
Data memory
Read data
Write data
MemRead
Instruction [15–0]
16
Sign extend
32
6
ALU control
ALUOp
Instruction [20–16]
Instruction [15–11]
RegDst
1
M
u
x
0

回顾：理想的流水线

- 目标（Goal）：以极少的成本增加来提高吞吐率（在指令处理的情况下，指硬件成本）。
- 重复执行相同操作（Repetition of identical operations）
  - 相同的操作在大量不同的输入上重复执行（例如，所有洗衣任务都经历相同的步骤）。
- 重复执行相互独立的操作（Repetition of independent operations）
  - 不同的操作之间没有依赖关系。
- 均匀可划分的子操作（Uniformly partitionable suboperations）
  - 处理过程可以均匀地划分为多个具有相同延迟的子阶段，这些子阶段**不共享资源**
- 典型示例（Fitting examples）：
  - 汽车装配线、洗衣任务。

指令流水线：不是理想的流水线

- **相同操作 … 不是！**
  - **不同的指令 → 并非所有指令都需要相同的阶段**
  - 迫使不同指令通过相同的流水线阶段会导致**外部碎片化（external fragmentation）**，即某些流水线阶段在某些指令执行时处于空闲状态。

指令流水线：不是理想的流水线

- **相同操作 … 不是！**
  - **不同的指令 → 并非所有指令都需要相同的阶段**
  - 迫使不同指令通过相同的流水线阶段会导致**外部碎片化（external fragmentation）**，即某些流水线阶段在某些指令执行时处于空闲状态。

- **均匀的子操作 … 不是！**
  - **不同的流水线阶段 → 不同的延迟**
  - 每个阶段需要通过同一个时钟来控制，这会导致**内部碎片化（internal fragmentation）**，即某些流水线阶段执行太快，而其他阶段的执行时间则相同，造成效率不平衡。

指令流水线：不是理想的流水线

- **相同操作 … 不是！**
  - **不同的指令 → 并非所有指令都需要相同的阶段**
  - 迫使不同指令通过相同的流水线阶段会导致**外部碎片化（external fragmentation）**，即某些流水线阶段在某些指令执行时处于空闲状态。

- **均匀的子操作 … 不是！**
  - **不同的流水线阶段 → 不同的延迟**
  - 每个阶段需要通过同一个时钟来控制，这会导致**内部碎片化（internal fragmentation）**，即某些流水线阶段执行太快，而其他阶段的执行时间则相同，造成效率不平衡。

- **相互独立的操作 … 不是！**
  - **指令并非相互独立**
  - 需要检测并解决指令之间的依赖关系，以确保流水线能够提供正确的结果。否则可能会导致**流水线停顿（pipeline stalls）**，即流水线并不总是处于运动状态。

流水线设计中的问题

- 平衡流水线各阶段的工作
  - 有多少个阶段，每个阶段做什么？
- 保持流水线的正确性、流动性和完整性，尤其在出现干扰流水线流动的事件时（Keeping the pipeline correct, moving, and full in the presence of events that disrupt pipeline flow）
  - 处理依赖关系（Handling dependencies）
    - 数据依赖（Data）
    - 控制依赖（Control）
  - 处理资源争用（Handling resource contention）
  - 处理长延迟（多周期）操作（Handling long-latency (multi-cycle) operations）
- 处理异常和中断（Handling exceptions, interrupts）
- 进阶：提高流水线吞吐量
  - 最小化流水线停顿（Minimizing stalls）

流水线停顿的原因（Pipeline Stalls）

- 停顿（Stall）：流水线停止运动的状态。
- 资源争用（Resource contention）
- 依赖关系（Dependences）（指令之间）
  - 数据（Data）
  - 控制（Control）
- 长延迟（多周期）操作（Long-latency (multi-cycle) operations）

依赖关系及其类型

- 也叫做“依赖性”或 “冒险”（hazard）。
- 依赖关系决定了指令之间的执行顺序要求。
- 有两种类型：
  - 数据依赖（Data dependence）
  - 控制依赖（Control dependence）
- 资源争用有时被称为资源依赖（Resource dependence）。
  - 然而，这并不是程序语义上根本的要求，因此我们将单独处理它。

处理资源争用（Resource Contention）

- 资源争用发生在两条流水线指令需要相同资源时。

处理资源争用（Resource Contention）

- 资源争用发生在两条流水线指令需要相同资源时。

- 解决方案 1：消除争用的根源
  - 复制资源或增加其吞吐量
    - 例如，使用独立的指令和数据存储器（缓存）
    - 例如，使用多个内存端口。

处理资源争用（Resource Contention）

- 资源争用发生在两条流水线指令需要相同资源时。

- 解决方案 1：消除争用的根源
  - 复制资源或增加其吞吐量
    - 例如，使用独立的指令和数据存储器（缓存）
    - 例如，使用多个内存端口。

- 解决方案 2：检测资源争用并停顿其中一个争用阶段
  - 你应该停顿哪个阶段？
  - 例如：如果寄存器文件只有一个读写端口，会怎样？

数据依赖（Data Dependences）

- **数据依赖的类型：**
  - 流依赖（Flow dependence，真实数据依赖——读后写）
  - 输出依赖（Output dependence，写后写）
  - 反依赖（Anti dependence，写后读）

- **哪些类型会在流水线机器中引起停顿？**
  - 对所有依赖，我们需要确保程序的语义是正确的。
  - 流依赖必须始终遵守，因为它们是对值的真实依赖。
  - 反依赖和输出依赖是由于**有限数量的架构寄存器**导致的。
    - 它们依赖的是名字，而不是值。
    - 我们稍后将讨论如何解决这些问题。

数据依赖类型

Flow dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4



Read-after-Write (RAW)

数据依赖类型

Flow dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

Read-after-Write (RAW)

Anti dependence

$r_3 \leftarrow r_1$ op r_2

$r_1 \leftarrow r_4$ op r_5

Write-after-Read (WAR)

数据依赖类型

Flow dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

Read-after-Write (RAW)

Anti dependence

$r_3 \leftarrow r_1$ op r_2

$r_1 \leftarrow r_4$ op r_5

Write-after-Read (WAR)

Output-dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

$r_3 \leftarrow r_6$ op r_7

Write-after-Write (WAW)

数据依赖类型

Flow dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

Read-after-Write (RAW)

Anti dependence

$r_3 \leftarrow r_1$ op r_2

$r_1 \leftarrow r_4$ op r_5

Write-after-Read (WAR)

Output-dependence

$r_3 \leftarrow r_1$ op r_2

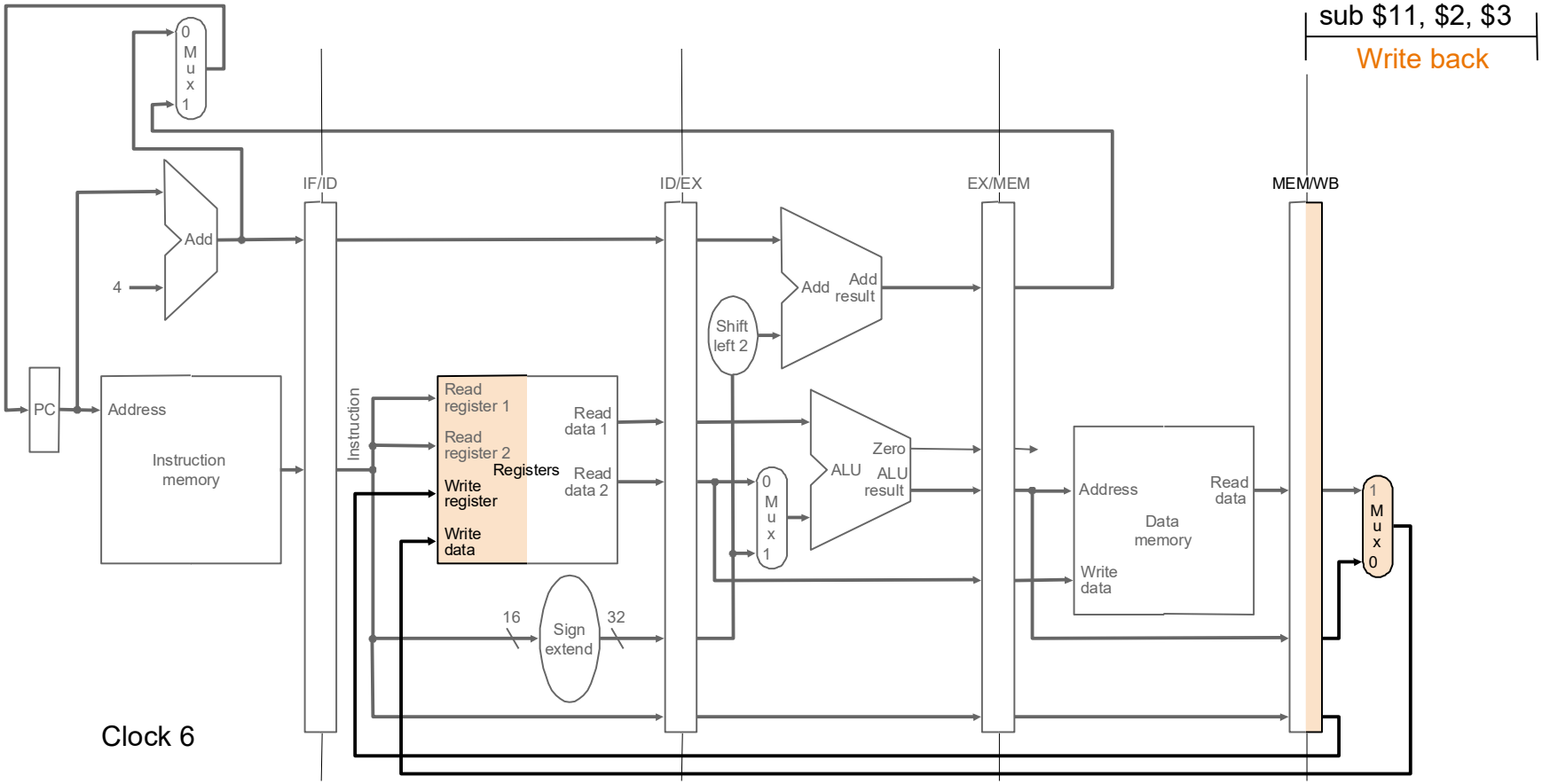
$r_5 \leftarrow r_3$ op r_4

$r_3 \leftarrow r_6$ op r_7

Write-after-Write (WAW)

寄存器重命名：解决依赖冲突

流水线操作示例


sub $11, $2, $3
Write back
IF/ID
ID/EX
EX/MEM
MEM/WB
0
M
u
x
1
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
16
Sign extend
32
1
M
u
x
0
Clock 6

今天的内容

- 处理数据依赖

如何处理数据依赖

- **反依赖和输出依赖 比较容易处理：**
  - 在一个阶段写入目标并按程序顺序执行。
- **流依赖 更复杂**
- **处理流依赖的五种基本方法：**
  - **检测并等待**，直到值在寄存器文件中可用。
  - **检测并转发/绕过** 数据到依赖的指令。
  - **在软件层面消除依赖**：无需硬件检测依赖。
  - **预测所需的值**，进行“猜测性”执行，并验证。
  - **做其他事情**（细粒度的多线程）：不需要检测。

互锁（Interlocking）

- 流水线处理器中指令之间依赖的检测，以保证正确执行。
- 基于软件的互锁 与 基于硬件的互锁。
- MIPS的缩写是什么？
  - “无内部互锁流水级的微处理器”(Microprocessor without interlocked piped stages)

依赖检测方法 I

- **记分板（Scoreboarding）：**
  - 每个寄存器都有一个有效位（Valid bit）与之关联。
  - 写入寄存器的指令会重置有效位。
  - 解码阶段的指令检查其所有源寄存器和目标寄存器是否有效。
    - 是：无需停顿……没有依赖。
    - 否：停顿指令。

- **优点：简单，每个寄存器1个位。**

- **缺点：需要对所有类型的依赖进行停顿，而不仅仅是流依赖。**

依赖检测方法 II

- **组合依赖检测逻辑（Combinational dependence check logic）：**
  - 特殊的逻辑，用来检测解码阶段的指令是否有后续指令写入该源寄存器。
    - 是：停顿指令/流水线。
    - 否：无需停顿……没有流依赖。

- **优点：不需要对反依赖和输出依赖进行停顿。**

- **缺点：逻辑比记分板复杂。**
  - 随着流水线的加深和加宽，逻辑会变得更加复杂（例如，考虑超标量执行）。

一旦在硬件中检测到依赖，怎么办？

- 选择：
  - 立即停顿依赖指令。
  - 仅在必要时停顿依赖指令——数据转发/绕过。
  - 选择其他方式…… （乱序执行）